

Laboratorio 2: Tiny MC

Computación Paralela - FAMAF

Javier Cremona
Marcelo Castillo

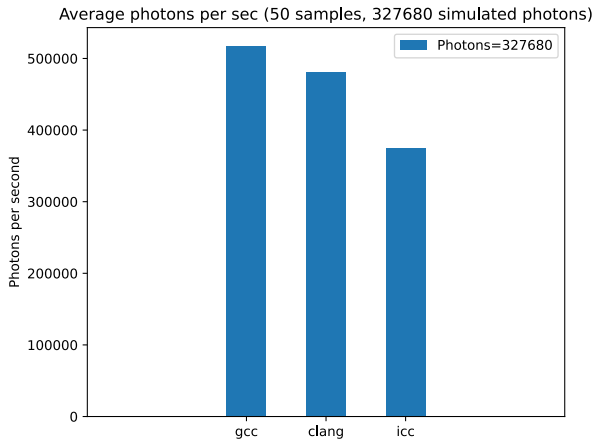
May 12, 2021

Cambios respecto al Laboratorio 1

En relación a lo hecho en el Laboratorio 1, se realizaron dos modificaciones:

- Reemplazamos el PRNG que usábamos por PCG.
- Agregamos el compilador de Intel icc a la lista de compiladores a usar.

Optimizaciones: O3, -march=native, -ffast-math, FDO

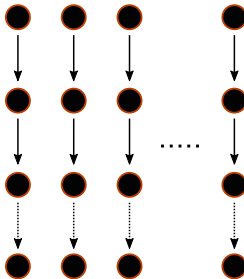


Vectorización en TinyMC

Sequential
Implementation



SIMD



Modificación Inicial:

- N generadores de números aleatorios independientes. Lo convertimos en Struct of Array.

Problemas:

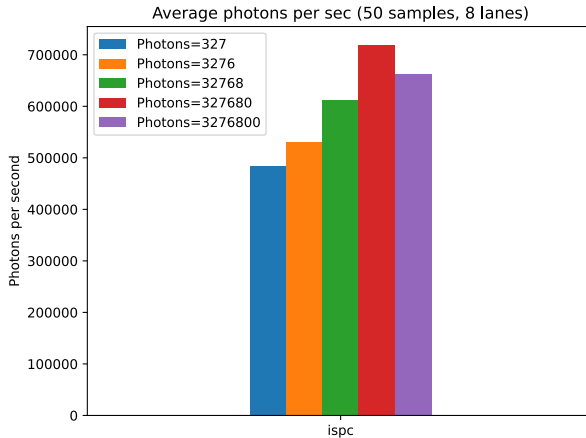
- Los fotones no terminan todos al mismo tiempo.
- Hay bucles para los que no se sabe en cuántas iteraciones van a finalizar.

Probamos distintas implementaciones:

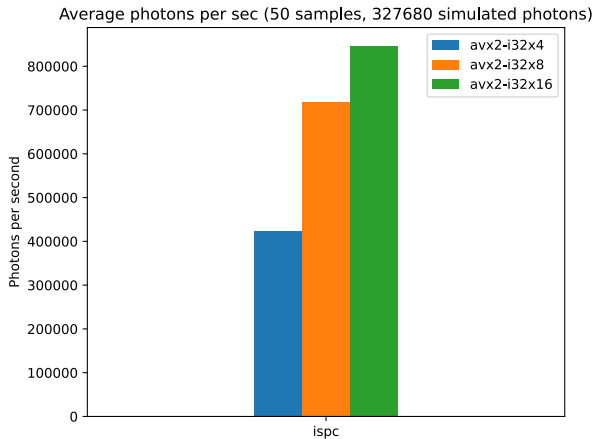
- 1 Implementamos todo en ispc.
- 2 Modificamos el método `photon()` buscando que el compilador autovectorice. Pedimos números aleatorios antes de usarlo en el bucle principal (lo llamamos `rand-offline`).
- 3 Modificamos el método `photon()` buscando que el compilador autovectorice. Pedimos números aleatorios cuando se necesitan en el bucle principal (lo llamamos `rand-ondemand`).

- Implementamos PCG en ISPC.
- Movimos el for que cuenta la cantidad de fotones dentro de la nueva implementación de `photon()`.
- En lugar de frenar un carril cuando finaliza la simulación de un `photon()`, reseteamos ese carril, y llevamos la cuenta de cuántos fotones llevamos simulados.
- Llevamos N arreglos `heat` y `heat2` para evitar race conditions, al finalizar toda la simulación juntamos todo en único arreglo y lo retornamos a C.

Caso 1: ISPC



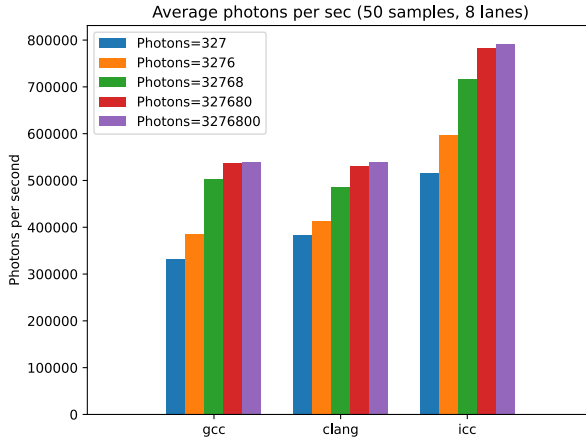
Caso 1: ISPC



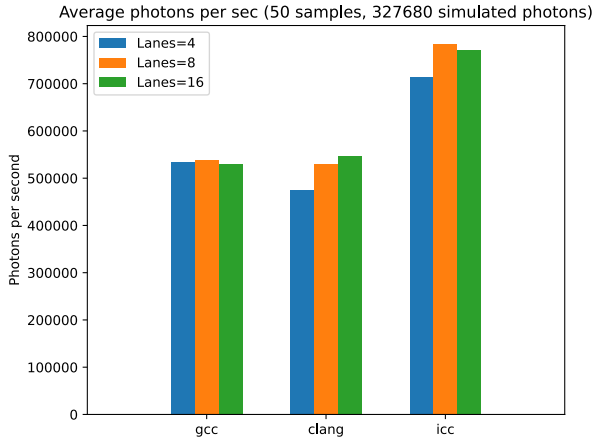
Caso 2: rand-offline

- Convertimos las variables escalares del método `photon()` en arreglos de `N` elementos.
- Movimos el `for` que cuenta la cantidad de fotones dentro de la nueva implementación de `photon()`.
- En lugar de frenar un carril cuando finaliza la simulación de un `photon()`, reseteamos ese carril, y llevamos la cuenta de cuántos fotones llevamos simulados.
- Los números aleatorios que se van a usar en una iteración del bucle principal se piden antes de iniciar una nueva iteración. Así evitamos frenar los carriles dentro del bucle principal.
- Al final de cada iteración, acumulamos el calor en la capa correspondiente.

Caso 2: rand-offline



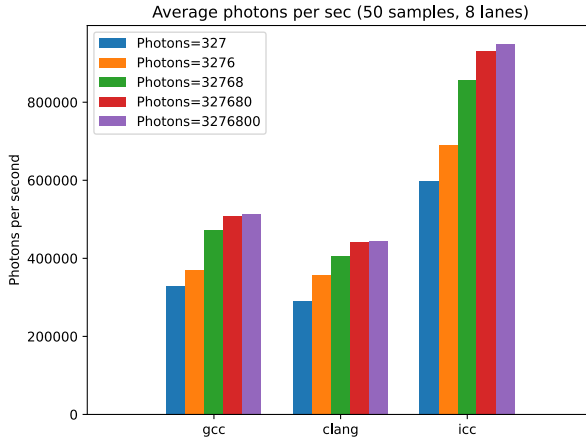
Caso 2: rand-offline



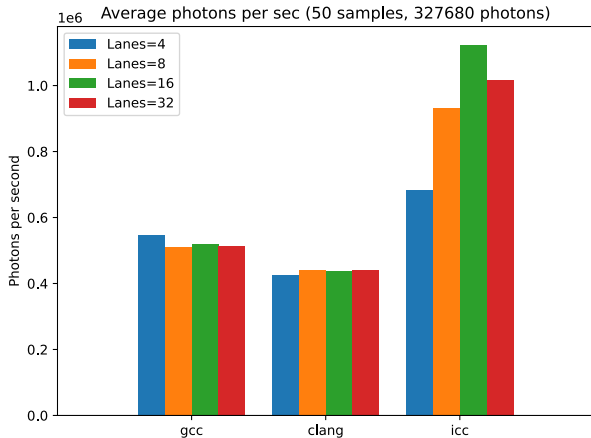
Caso 3: rand-ondemand

- Similar al Caso 2, excepto que aquí los números aleatorios se piden dentro del bucle.
- Llevamos una máscara durante toda la ejecución del bucle. Si el par de números aleatorios no cumple con la condición de pertenecer al círculo unitario, se para ese único carril usando la máscara. Así evitamos iterar con un inner loop (do-while) para el cual no sabemos cuántas iteraciones va a realizar.

Caso 3: rand-ondemand



Caso 3: rand-ondemand



- En Sandy Bridge, prácticamente no hay diferencia entre los casos 2 y 3.
- En Sandy Bridge, icc vectoriza loops con O2 que con O3 no puede vectorizar.
- En varios casos, tanto en Sandy Bridge como en Broadwell, se observaron mejores resultados con 16 lanes... aunque no tenemos AVX512!

- Probamos diferentes implementaciones.
- El código de ispc es más legible.
- Para los casos 2 y 3, se requirió mayor esfuerzo del programador.
- icc fue más eficiente al autovectorizar.